

Reinforcement Learning

Finite Markov Decision Processes

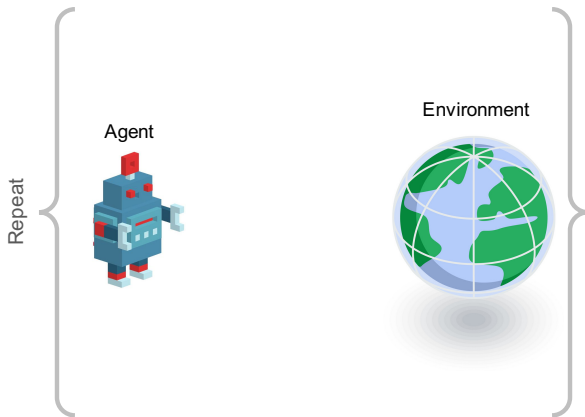
Min-hwan Oh

Seoul National University

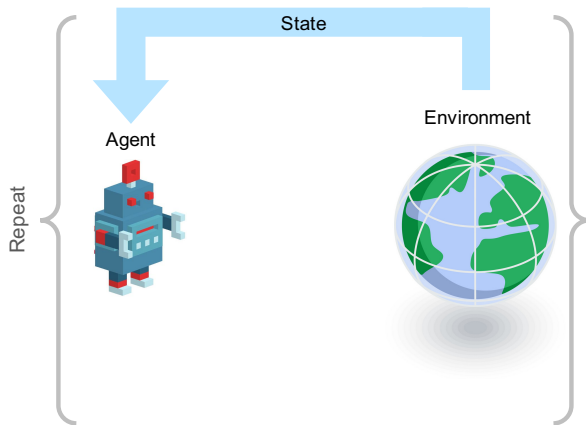
Markov Decision Processes (MDPs)

- Classical formalization of sequential decision making
- Actions influence not just immediate rewards, but also subsequent states (contrast to multi-armed bandits)

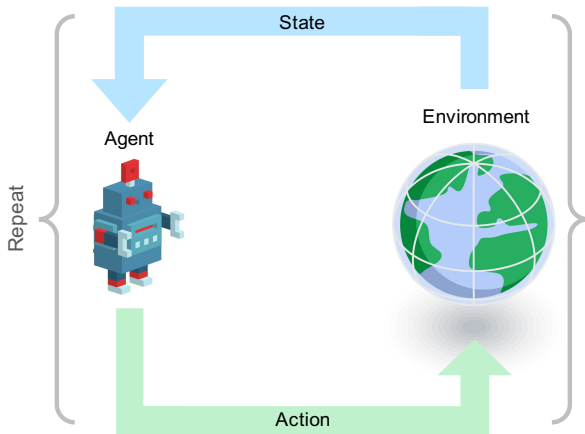
Agent-Environment Interface



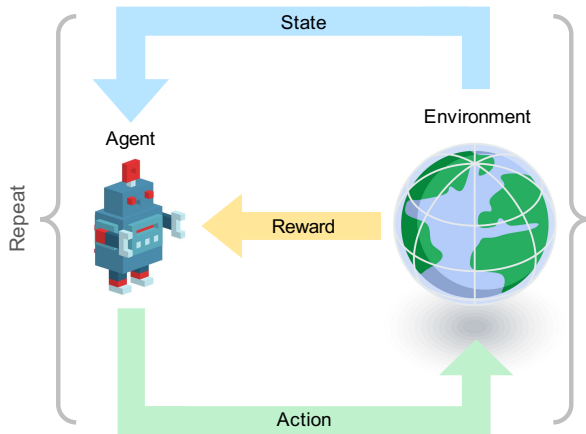
Agent-Environment Interface



Agent-Environment Interface



Agent-Environment Interface



Agent-Environment Interface

For each time $t = 0, 1, 2, \dots$,

1. agent receives state information $S_t \in \mathcal{S}$
2. agent selects an action $A_t \in \mathcal{A}(s_t)$
3. (one step later) agent receives reward R_{t+1} and next state S_{t+1} is determined.

Interactions produce a sequence (or trajectory)

- $S_0, A_0, R_1, S_1, A_1, R_2, A_2, R_3, \dots$

Dynamics of MDP

- Dynamics function $p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$
 - ▶ takes 4 arguments: next state, reward, current state, action
- Given particular state s and action a , probability of next state and reward is

$$p(s', r \mid s, a) := \mathbb{P} \{ S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a \}$$

for all $s', s \in \mathcal{S}, r \in \mathcal{R}$, and $a \in \mathcal{A}(s)$.

Markov Property

- “The future is independent of the past given the present.”
- A state s has the **Markov** property if

$$\begin{aligned}\mathbb{P}\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\} \\ = \mathbb{P}\{S_t = s', R_t = r \mid S_0, A_0, \dots, S_{t-1} = s, A_{t-1} = a\}\end{aligned}$$

for all possible histories $\{S_0, A_0, \dots, S_{t-1}, A_{t-1}\}$.

- ▶ State captures all relevant information from history.
- ▶ Once state is known, entire history is not needed.
- ▶ State is a sufficient statistic of the past.

State-Transition and Reward Models

Can compute using dynamics $p(s', r \mid s, a)$

- State-transition

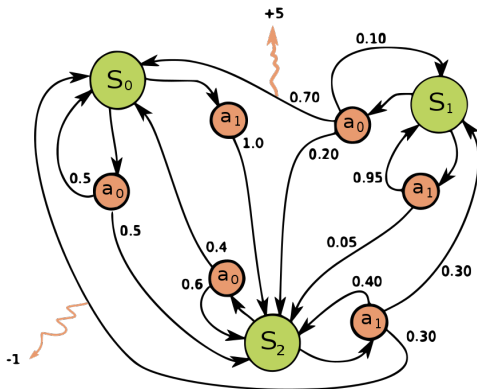
$$p(s' \mid s, a) := \mathbb{P} \{ S_t = s' \mid S_{t-1} = s, A_{t-1} = a \} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$$

- Expected rewards for state-action pair

$$r(s, a) := \mathbb{E} \{ R_t \mid S_{t-1} = s, A_{t-1} = a \} = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$$

- Expected rewards for state-action-next-state triple

$$r(s, a, s') := \mathbb{E} \{ R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s' \} = \sum_{r \in \mathcal{R}} r \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}$$



- Joint probability of s_0 and reward 5 after taking a_0 in s_1 :
 $p(s_0, 5 | s_1, a_0) = 0.7$
- Transition to s_2 after taking a_0 in s_1 : $p(s_2 | s_1, a_0) = 0.2$
- Expected reward of a_0 in s_1 : $r(s_1, a_0) = 5 \cdot 0.7 + 0 \cdot 0.1 + 0 \cdot 0.2 = 3.5$

Boundary between Agent and Environment

- Boundary in the MDP framework: not (necessarily) the same as the physical boundary of an agent (e.g., human, robot)
- Muscles, skeleton: part of the environment
- Environment in MDP framework: anything that cannot be changed arbitrarily by agent

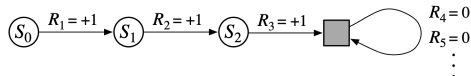
Reward Hypothesis

“The goal of the agent is the maximization of the expected value of the cumulative sum of a received scalar (reward) signal.”

- Rewards need to truly indicate what needs to be accomplished (goal).
- Question: Is setting rewards for sub-goals a good idea?

Returns

- Goal: maximize cumulative reward the agent receives in long run
- (Formally) maximize the **expected return** G_t
 - ▶ (Episodic) $G_t := R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$
 - ▶ (Continuing) $G_t := R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ where $\gamma \in [0, 1]$ is the discount rate.
- Can combine episodic and continuing tasks by considering the **absorbing state**



- ▶ We can write $G_t := \sum_{k=t+1}^T \gamma^{k-t-1} R_k$ (possibly $T = \infty$ or $\gamma = 1$)

Policies and Value Functions

- Policy: mapping from states to possible action
 - ▶ $\pi(a|s)$ at time t : probability that $A_t = a$ given $S_t = s$
 - ▶ Deterministic policy $\pi(a'|s) = 1$ for some a' and $\pi(a|s) = 0$ for all $a \neq a'$
- Value function: estimates how good state (or state-action pair) is
 - ▶ State-value function under policy π

$$v_{\pi}(s) := \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

- ▶ Action-value function under policy π

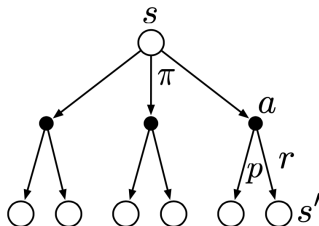
$$q_{\pi}(s, a) := \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

Bellman Equation

$$\begin{aligned}v_{\pi}(s) &:= \mathbb{E}_{\pi} [G_t \mid S_t = s] \\&= \mathbb{E}_{\pi} [R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\&= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) [r + \gamma \mathbb{E}_{\pi} [G_{t+1} | S_{t+1} = s']] \\&= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) [r + \gamma v_{\pi}(s')]\end{aligned}$$

- Bellman equation: shows relationship between the value of a state and the values of its successor states

Bellman Backups



$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) [r + \gamma v_{\pi}(s')]$$

- Averages over all the possibilities of subsequent states and rewards
- Transfer value information of leaf states back to the root state

Example: Gridworld (with Wormholes)

- Actions: north, south, east, and west
- Rewards: **+10** for A to A', **+5** for B to B', **-1** for going off the grid
- State transition: deterministic 4 direction move except A to B
 - ▶ All 4 actions in A and B lead to A' and B' respectively

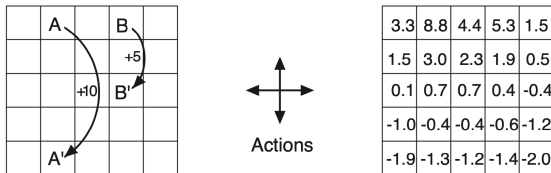


Figure: Gridworld: exceptional reward dynamics (left) and state-value function for the uniform random policy (right)

Optimal Policies and Optimal Value Functions

- A policy π is better than π' if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$.
- **Optimal policy**: better than or equal to all other policies
- Optimal **state-value** function:

$$v_*(s) := \max_{\pi} v_{\pi}(s) \quad \text{for all } s \in \mathcal{S}$$

- Optimal **action-value** function:

$$q_*(s, a) := \max_{\pi} q_{\pi}(s, a) \quad \text{for all } s \in \mathcal{S}, a \in \mathcal{A}(s)$$

- can write q_* in terms of v_*

$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

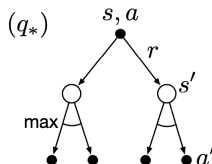
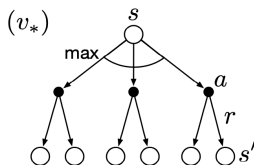
Bellman Optimality Equation

- Bellman equation for v_*

$$\begin{aligned}v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\&= \max_a \mathbb{E}_{\pi_*} [G_t \mid S_t = s, A_t = a] \\&= \max_a \mathbb{E} [R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\&= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]\end{aligned}$$

- “The value of a state under an optimal policy must equal the expected return for the best action from that state.”

Bellman Optimality (Continued)

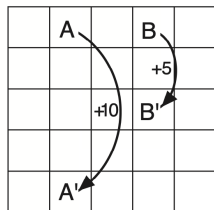


- Difference from Bellman backup diagram: max operations
- Bellman optimality equation has a unique solution for v_* .

$$v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$$

- Once one has v_* , it is easy to determine an optimal policy.
 - ▶ any policy greedy with respect to v_* is an optimal policy!
 - ▶ having q_* is easier for finding optimal policy

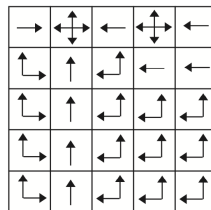
Gridworld: Optimal Value Function and Policy



Gridworld

22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

v_*



π_*

Caveat

The optimal solution we have seen relies on at least the following three assumptions

1. The dynamics of the environment are accurately known
2. Computational resources are sufficient to complete the calculation
3. States have the Markov property.

In practice, some kind of approximate solution is needed (will learn this later in the course).

Next class: Dynamic Programming (Chapter 4 of Sutton & Barto)